

Inference Acceleration as Closed-Loop Perturbation: Closed-Loop Sensitivity-Guided Tactic Search for VLA Policies

patrick.zhang
patrick.zhang5233@gmail.com

Abstract

Inference acceleration for vision-language-action (VLA) policies is usually framed as a local systems problem: quantize weights, compile graphs, fuse kernels, or replay static execution paths, then check numerical drift and serving latency. This framing misses the central difficulty for robot policies. The accelerated model is not only an approximator of a fixed function; it is a controller inside a feedback loop. A small implementation perturbation can change contact timing, future observations, receding-horizon replanning, and ultimately whether a rollout crosses a thresholded success margin.

We formulate post-training VLA acceleration as a closed-loop policy perturbation problem and propose Closed-Loop Sensitivity-Guided Tactic Search (CLSG-TS), a practical procedure for selecting acceleration tactics under rollout budget. The method first uses cheap static and open-loop filters to prune candidate tactics, then evaluates the surviving candidates on matched fragile rollouts, estimates closed-loop risk from paired repairs/regressions and first-divergence diagnostics, and finally selects a behavior-first or speed-constrained deployment point on a held-out speed-risk frontier.

This framework yields five claims that organize the study: implementation error is filtered by closed-loop sensitivity; rollout flips are margin-crossing events; fixed-distribution validation is insufficient without state-distribution control; sensitivity is anisotropic across action channels, rollout phases, and model boundaries; and acceleration is best understood as constrained exploration of nearby policy implementations rather than imitation of one FP16 point. We test these claims on GR00T policies in LIBERO-10 using `torch.compile` action-head acceleration, eager-island and duration fallback tactics, controlled action perturbations down to 10^{-6} action magnitude, same-observation backend-drift replay, first-divergence traces, matched repair/regression accounting, and held-out active tactic search.

The empirical picture is non-monotonic. Speed-only action-head compilation gives a $2.23\times$ median server-latency speedup on a 33-case GR00T N1.5 probe, yet reduces success from 19/33 to 16/33. CLSG-TS recovers behavior without simply reverting to FP16: an early-block eager island reaches 18/33 at $2.31\times$ speedup, and a fine duration probe finds that protecting policy steps 0–120 restores 19/33 at 69.66 ms versus 156.22 ms for FP16. Held-out validation then changes the conclusion from “choose the local winner” to “choose a point on a speed-risk frontier.” Across two N1.5 30-case folds, speed-only execution is fastest but has 7 paired regressions; the 0–120 duration tactic and a combined early-block plus 0–120 tactic both reach 47/60 pooled success with different speed-risk trade-offs. On GR00T N1.7, we instantiate CLSG-TS as a small Bayesian/active search over duration-window tactics: a warm-start probe and one BO batch identify a behavior-first 0–20 window and a speed-constrained 2–12 window, which are then validated on new initial states.

We do not claim universal superiority of any single backend or tactic. The contribution is a method for VLA acceleration under closed-loop risk: estimate which perturbations are sensitive, then select compilation, fallback, kernel, quantization, or routing tactics by matched rollout repair/regression and latency rather than by one-step drift or a single probe winner.

1 Introduction

Large vision-language-action models are increasingly used as generalist robot policies. They map visual observations and language instructions into action chunks, often through diffusion-style denoising or transformer action heads. As these models grow, inference acceleration becomes unavoidable: deployment systems will quantize weights, compile graphs, fuse kernels, cache repeated structure, replay static CUDA graphs, or mix

high- and low-precision execution. The usual evaluation asks whether the accelerated implementation is numerically close to the reference policy and faster to serve.

For closed-loop robot policies, numerical closeness is not the whole problem. An action difference at time t changes the next observation. The next action is then produced on a state that may not have been visited by the original FP16 policy. In manipulation, this feedback loop can change contact timing, grasp stability, object pose, or whether the terminal state crosses a binary success predicate. A tiny implementation difference can be absorbed in one phase, amplified near a contact boundary, or even repair a failure by nudging the rollout into a different trajectory basin.

This paper studies the following question:

How should VLA inference acceleration be designed and evaluated when the accelerated implementation is part of a closed-loop control system?

Our answer is that post-training acceleration should be treated as a structured policy perturbation. Quantization, graph compilation, eager islands, and kernel replacement are different implementation maps from a reference policy to a constrained, faster policy. The relevant object is not only the one-step action drift on fixed observations, but also how the implementation redistributes success and failure over matched closed-loop rollouts. This changes the goal from finding a single numerically closest implementation to mapping a speed-behavior design space.

We organize the paper around five claims:

1. **Closed-loop filtering:** acceleration-induced action error matters through dynamics, policy feedback, and future closed-loop gain, not only through its norm.
2. **Margin and basin sensitivity:** rollout flips occur when perturbations cross task success margins or trajectory-basin boundaries.
3. **Open-loop insufficiency:** fixed-distribution action drift cannot predict behavior under the accelerated policy’s own induced state distribution.
4. **Anisotropic sensitivity:** action channels, rollout durations, and model-layer boundaries are not equally sensitive.
5. **Solution-space exploration:** acceleration can create repairs and regressions, so the useful target is not blind FP16 imitation but sensitivity-guided search for faster implementations with better paired repair/regression trade-offs.

The paper follows a single arc. We first derive why local implementation error must be weighted by closed-loop dynamics and task margins. We then identify where perturbations matter most across action dimensions, rollout phases, and execution boundaries. Finally, we turn those sensitivity maps into CLSG-TS and validate the resulting speed-risk frontier on held-out folds and a newer checkpoint. Our contributions are:

1. We formulate VLA inference acceleration as a closed-loop policy perturbation problem and derive local propagation, margin-crossing, and distribution-shift expressions that explain why local drift is not sufficient.
2. We propose CLSG-TS, a rollout-budget-aware tactic-search workflow that combines cheap filters, fragile matched probes, first-divergence diagnostics, paired repair/regression accounting, and held-out speed-risk selection.
3. We show that action channels, rollout phases, and action-head execution boundaries have strongly anisotropic closed-loop sensitivity, so equal-size implementation perturbations can have different behavioral effects.
4. We show that speed-only action-head compilation can be fast but behaviorally unsafe, and that sensitivity-guided eager islands and duration fallbacks can recover behavior while preserving much of the latency gain.
5. We show that a strong local tactic, policy steps 0–120, is not universal: multi-fold validation and an N1.7 CLSG-BO check change the conclusion from selecting one fixed winner to selecting behavior-first or speed-constrained tactics by held-out success, paired regressions, and latency.

From point solution to design-space map. The contribution is not a single deployment backend. We use compilation, fallback, and routing probes to expose the geometry of the problem: which perturbations are absorbed, which are amplified, which rollouts are repaired or regressed, and which dimensions, phases, or modules should be protected. The intended output is therefore a map of the acceleration design space and a tactic-selection procedure, not only a point estimate for one checkpoint.

Model-class interpretation. Our findings are not tied to a specific checkpoint-level failure pattern. They suggest a model-class behavior: for DiT-style VLA or world-action models, small acceleration-induced implementation perturbations are filtered through closed-loop sensitivity, task margins, and trajectory basins. Larger or newer GR00T-style variants, as well as related action-diffusion policies, should therefore be evaluated by re-estimating their sensitivity maps rather than assuming uniform acceleration tolerance across dimensions, time, or layers.

Scope of claims. This paper should be read as a behavior-level modeling and tactic-selection study, not as a new compiler or production deployment benchmark. The acceleration track uses prototype action-head execution boundaries and fallback windows. We do not claim final memory, energy, or production backend performance. We also do not treat small aggregate success-rate gaps as policy dominance. The main claim is methodological: VLA acceleration should be evaluated and designed as sensitivity-weighted closed-loop perturbation, using matched rollouts and paired repair/regression analysis rather than only average open-loop action error or raw serving latency.

Experimental roadmap. The experiments are intentionally staged so that each stage answers the question raised by the previous one. First, controlled action perturbations, same-observation backend-drift replay, and layer/boundary probes identify sensitive directions, rollout windows, and execution boundaries. Second, mixed eager/compiled acceleration policies test whether those maps can guide a faster implementation. Third, held-out folds and the N1.7 CLSG-BO check show why the final object should be a robust tactic-selection protocol rather than a single fixed window.

2 Related Work

Vision-language-action policies. Recent robot policies increasingly combine large-scale visual, language, and action modeling. RT-1 introduced a scalable transformer policy trained on diverse real-robot data [2]. RT-2 framed robotic actions as tokens inside a vision-language-action model and showed transfer from web-scale vision-language training to control [3]. OpenVLA made this direction more accessible by releasing a 7B-parameter open-source VLA trained on large robot demonstration mixtures, and also noted practical interest in efficient fine-tuning and quantized serving [7]. GR00T N1 uses a dual-system VLA architecture where a vision-language module conditions a diffusion transformer action module [10]. Diffusion Policy and related action-diffusion methods model robot behavior as conditional denoising, often with receding-horizon control [4]. Our work focuses on the evaluation problem that appears when such policies are compressed or accelerated after training.

Inference acceleration for closed-loop policies. Deployment acceleration can come from low-precision arithmetic, graph compilation, operator fusion, CUDA graph replay, caching, or custom kernels. In open-loop perception or language tasks, small numerical differences are usually judged by output agreement and throughput. In VLA policies, the accelerated implementation chooses actions that alter future observations. This makes acceleration a behavioral question as well as a systems question: a faster implementation is useful only if its closed-loop policy perturbation stays within acceptable task margins.

Post-training acceleration and quantization. Post-training acceleration includes graph compilation, operator fusion, kernel replacement, mixed precision, and low-bit quantization. PTQ methods for large transformers often target memory and latency while preserving model quality. LLM.int8() handles transformer outliers through mixed-precision decomposition [5]; GPTQ uses approximate second-order information for one-shot low-bit weight quantization [6]; SmoothQuant migrates quantization difficulty from activations

to weights through an equivalent smoothing transform [12]; and AWQ uses activation statistics to protect salient weight channels [8]. QuantVLA is the most direct prior work on PTQ for VLA systems, proposing scale-calibrated selective quantization, attention temperature matching, and output head balancing [13]. These systems motivate our setting, but our contribution is not another quantizer. We study the closed-loop tactic-selection problem that arises for any post-training implementation change, including compilation, fallback routing, custom kernels, and future low-precision backends.

Sequential prediction and distribution shift. The gap between offline error and closed-loop behavior is a classic issue in imitation learning and sequential prediction. DAgger formalizes the problem that future observations depend on previous actions, so policies trained or evaluated only under a fixed data distribution can suffer from compounding errors [11]. VLA acceleration creates a related but distinct problem: the policy is already trained, but a post-training implementation change perturbs its action outputs and therefore perturbs the state distribution it induces. This makes paired rollout analysis a natural complement to fixed-distribution action-drift checks.

Feedback control perspective. Classical feedback control emphasizes that closed-loop behavior is determined by the interaction between controller perturbations, plant dynamics, and feedback gain [1]. We use this lens to interpret inference acceleration not as a static approximation problem, but as a bounded policy perturbation whose effect depends on local closed-loop sensitivity and task margins.

3 Inference Acceleration as Policy Perturbation

Let $\pi_\theta(a \mid s, l)$ denote a reference FP16 VLA policy conditioned on state observation s and language instruction l . A post-training acceleration transform constructs an implemented policy

$$\varphi = \mathcal{A}(\theta; C, S), \quad \pi_\varphi = \pi_\varphi, \quad (1)$$

where $\mathcal{A}$ may represent graph compilation, mixed-precision partitioning, kernel replacement, graph replay, fallback routing, or quantization; C denotes calibration or profiling data when applicable; and S denotes systems choices such as compilation boundaries, fallback windows, tactic selectors, or shape buckets. Even without gradient updates, this is not a neutral storage transform: it changes the implemented function.

Offline action drift measures a local difference such as

$$e(s) = \|\pi_\varphi(s, l) - \pi_\theta(s, l)\| \quad (2)$$

on observations sampled from some dataset or reference rollout distribution. Closed-loop performance depends on the state distributions induced by the two policies, d_{π_θ} and d_{π_φ} .

For a local transition model,

$$s_{t+1} = f(s_t, a_t), \quad a_t = \pi(s_t), \quad (3)$$

a first-order error expansion gives

$$\delta s_{t+1} \approx J_s \delta s_t + J_a \delta a_t, \quad \delta a_t = \pi_\varphi(s_t) - \pi_\theta(s_t). \quad (4)$$

Even if δa_t is small on the reference rollout distribution, the accumulated δs_t can move the accelerated policy into states where the original local error estimate no longer applies.

This is why manipulation success is not a smooth function of single-step action MSE. A rollout can flip because of a small action perturbation if that perturbation changes whether the gripper reaches the object before closing, whether a placed object crosses a success-region boundary, whether an object contact produces a different pose, or whether a receding-horizon action chunk enters a different trajectory basin. In this view, acceleration is closer to post-training policy editing than to passive systems optimization.

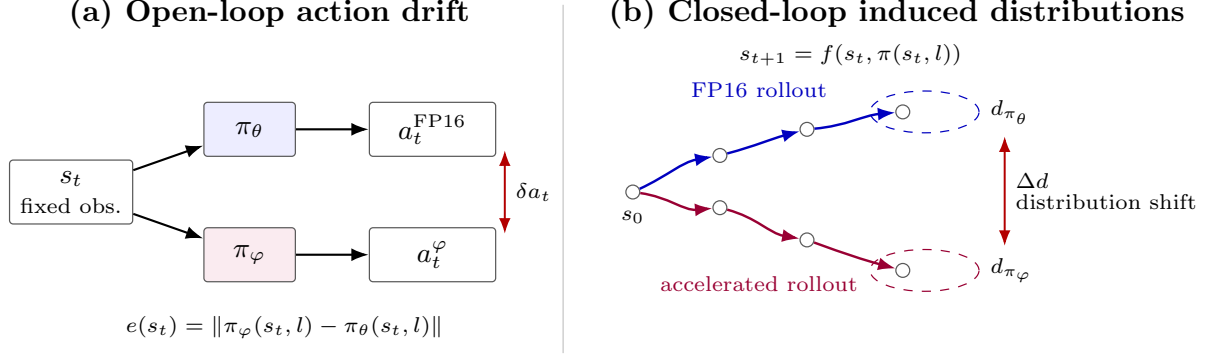


Figure 1: Inference acceleration as closed-loop policy perturbation. Offline action drift compares two actions on a fixed observation, while closed-loop execution lets each policy induce its own future observation distribution.

Claim 1: implementation error is filtered by closed-loop sensitivity. The first-order expression above can be expanded over a horizon. Let $\eta_t = \pi_\varphi(s_t, l) - \pi_\theta(s_t, l)$ be the local acceleration-induced action perturbation on the nominal FP16 trajectory, let $A_t = \partial f / \partial s$, $B_t = \partial f / \partial a$, and $K_t = \partial \pi_\theta / \partial s$ be evaluated along that trajectory, and define the local closed-loop Jacobian

$$F_t = A_t + B_t K_t. \quad (5)$$

This claim is local and first-order: it assumes small perturbations around the FP16 trajectory, defines η_t on that nominal trajectory, and uses the FP16 policy Jacobian as a nominal proxy. A more complete local recursion would include the accelerated policy’s feedback gain: with $K_t^\varphi = \partial \pi_\varphi / \partial s$ and $\Delta K_t = K_t^\varphi - K_t$, $\delta s_{t+1} \approx (F_t + B_t \Delta K_t) \delta s_t + B_t \eta_t$. The main surrogate omits this hard-to-estimate term; closed-loop validation captures the combined rollout effect of action offsets and feedback-gain changes empirically. Under the nominal-proxy assumptions, the terminal state perturbation satisfies

$$\delta s_T \approx \sum_{t=0}^{T-1} (F_{T-1} F_{T-2} \cdots F_{t+1}) B_t \eta_t, \quad (6)$$

with the empty product interpreted as the identity. Thus the effect of acceleration is not determined by $\|\eta_t\|$ alone, but by how its direction aligns with transition dynamics, policy feedback, and future closed-loop gain.

Claim 2: rollout flips are margin-crossing events. Let $h(s_T)$ be an implicit terminal success margin, with success when $h(s_T) > 0$. For a small terminal perturbation,

$$h(s_T^q) - h(s_T) \approx \nabla h(s_T)^\top \delta s_T. \quad (7)$$

A rollout can flip when the projected perturbation is comparable to the FP16 margin:

$$|\nabla h(s_T)^\top \delta s_T| \gtrsim |h(s_T)|. \quad (8)$$

More precisely, the outcome flips when the perturbation changes the sign of the margin:

$$\text{sign}(h(s_T^q)) \neq \text{sign}(h(s_T)). \quad (9)$$

If the FP16 rollout succeeds, $h(s_T) > 0$, an accelerated regression requires

$$\nabla h(s_T)^\top \delta s_T \lesssim -h(s_T). \quad (10)$$

If the FP16 rollout fails, $h(s_T) < 0$, an accelerated repair requires

$$\nabla h(s_T)^\top \delta s_T \gtrsim |h(s_T)|. \quad (11)$$

Thus Equation 8 should be read as a magnitude condition for either flip direction; the sign determines whether the perturbation causes a repair or a regression. This explains why two rollouts with similar single-step action drift can behave differently: high-margin trajectories absorb the perturbation, while low-margin trajectories near contact or placement boundaries can cross the success threshold.

Design principle: optimize sensitivity-weighted closed-loop perturbation. Equations 6 and 8 suggest that the relevant post-training acceleration objective for VLA policies is not the average open-loop action deviation

$$\min_Q \mathbb{E} \sum_t \|\eta_t(Q)\|^2, \quad (12)$$

but a sensitivity-weighted perturbation objective of the form

$$\min_Q \mathbb{E} \sum_t \eta_t(Q)^\top G_t \eta_t(Q), \quad (13)$$

where one local terminal-margin approximation is

$$G_t = B_t^\top \Phi_{t \rightarrow T}^\top \nabla h(s_T) \nabla h(s_T)^\top \Phi_{t \rightarrow T} B_t, \quad \Phi_{t \rightarrow T} = F_{T-1} \cdots F_{t+1}. \quad (14)$$

Equation 13 is a separable local surrogate, not the exact squared full-horizon terminal-margin objective. To see this, define the scalar contribution

$$\alpha_t = \nabla h(s_T)^\top \Phi_{t \rightarrow T} B_t \eta_t = c_t^\top \eta_t, \quad c_t = B_t^\top \Phi_{t \rightarrow T}^\top \nabla h(s_T).$$

The first-order terminal margin perturbation is $\Delta h \approx \sum_t \alpha_t$. Its exact squared approximation expands as

$$(\Delta h)^2 \approx \left(\sum_t \alpha_t \right)^2 = \sum_t \alpha_t^2 + 2 \sum_{i < j} \alpha_i \alpha_j.$$

The per-step term is $\sum_t \alpha_t^2 = \sum_t \eta_t^\top c_t c_t^\top \eta_t$, which gives Equation 13 with $G_t = c_t c_t^\top$. The omitted cross-time terms represent interactions between implementation errors at different rollout steps: they can amplify risk when their margin effects have the same sign, or partially cancel when their effects have opposite signs. We use the separable form as a tractable proxy, appropriate when cross-time error correlations are weak or when the objective is meant to rank locally sensitive perturbation directions rather than exactly optimize rollout-level margin loss. In practice G_t is difficult to estimate exactly, but the expression identifies the target: reduce implementation error in state-action directions that are amplified by feedback dynamics and projected onto small task margins. CLSG-TS uses rollout diagnostics, paired repairs/regressions, and sensitivity probes as practical proxies for this closed-loop criterion.

Claim 3: open-loop drift is incomplete without state-distribution control. Let $\ell(s)$ denote a closed-loop-relevant loss, such as action drift, sensitivity-weighted action error, or risk of approaching a failure boundary. Offline validation estimates behavior under a fixed distribution, typically d_{π_θ} or a dataset distribution. Closed-loop accelerated execution instead samples from d_{π_φ} :

$$\mathbb{E}_{s \sim d_{\pi_\varphi}} [\ell(s)] = \mathbb{E}_{s \sim d_{\pi_\theta}} [\ell(s)] + \left(\mathbb{E}_{s \sim d_{\pi_\varphi}} [\ell(s)] - \mathbb{E}_{s \sim d_{\pi_\theta}} [\ell(s)] \right). \quad (15)$$

Open-loop action drift controls only the fixed-distribution term. It does not by itself control the distribution-shift term induced by executing the perturbed policy. This term can matter because a small nominal action error can move the system away from the FP16 trajectory, after which the accelerated policy may visit states with larger implementation error, higher closed-loop sensitivity, or smaller task margins.

More formally, if ℓ is a nonnegative bounded loss with $0 \leq \ell(s) \leq \|\ell\|_\infty$, then the distribution-shift contribution obeys

$$\left| \mathbb{E}_{d_{\pi_\varphi}} [\ell] - \mathbb{E}_{d_{\pi_\theta}} [\ell] \right| \leq \|\ell\|_\infty \text{TV}(d_{\pi_\varphi}, d_{\pi_\theta}), \quad (16)$$

under the standard definition $\text{TV}(p, q) = \frac{1}{2} \|p - q\|_1$. If ℓ is L -Lipschitz under a state metric, an analogous Wasserstein bound gives

$$\left| \mathbb{E}_{d_{\pi_\varphi}} [\ell] - \mathbb{E}_{d_{\pi_\theta}} [\ell] \right| \leq L W_1(d_{\pi_\varphi}, d_{\pi_\theta}). \quad (17)$$

The point is not that these distances are easy to estimate in high-dimensional robot rollouts, but that fixed-distribution drift is only one term. Predicting rollout robustness also requires controlling how far the accelerated policy moves the visited state distribution.

Claim 4: closed-loop sensitivity is anisotropic. The metric G_t implies that not all perturbations of the same norm are equally important. For a unit action coordinate e_j , an equal-magnitude perturbation $\eta_t = \epsilon e_j$ has local squared terminal-margin risk

$$r_{t,j} \approx \epsilon^2 e_j^\top G_t e_j. \quad (18)$$

Thus action channels are only equally sensitive in the special case where the relevant directional structure of G_t is uniform. In manipulation, this is unlikely: Cartesian motion, wrist orientation, and gripper timing couple differently to contact dynamics and task margins.

The same argument applies across time. For a fixed perturbation direction v injected at two rollout steps, the local risk is

$$r_t(v) \approx v^\top G_t v, \quad (19)$$

and G_t changes with B_t , $\Phi_{t \rightarrow T}$, and $\nabla h(s_T)$. Early approach, alignment, contact, grasping, transfer, and placement windows can therefore have different sensitivity even when the perturbation norm is unchanged.

Finally, for a module- or layer-level implementation perturbation $\zeta_{i,t}$ whose first-order effect on the action is $\eta_t \approx J_{i,t} \zeta_{i,t}$, the corresponding risk is

$$r_{i,t} \approx \zeta_{i,t}^\top J_{i,t}^\top G_t J_{i,t} \zeta_{i,t}. \quad (20)$$

This shows why acceleration choices cannot be based only on parameter count, FLOPs, or latency. A module, graph boundary, or duration window is attractive only if it has high speed benefit and low sensitivity-weighted risk. The practical consequence is the design rule we test later: not all action dimensions, rollout durations, or layer boundaries are equal.

Claim 5: acceleration explores a constrained solution space. If the accelerated implementation induces a different state distribution, the goal cannot be simply to mimic every FP16 rollout. FP16 is a learned policy, not an oracle. A perturbation can repair an FP16 failure by moving the system into a better trajectory basin, or regress an FP16 success by crossing a fragile margin. Aggregate success differences decompose exactly into these two directions. For matched cases $c \in \mathcal{C}$ and binary success indicators $S_A(c), S_B(c) \in \{0, 1\}$,

$$\sum_{c \in \mathcal{C}} S_B(c) - \sum_{c \in \mathcal{C}} S_A(c) = \sum_{c \in \mathcal{C}} \mathbf{1}\{S_A(c) = 0, S_B(c) = 1\} - \sum_{c \in \mathcal{C}} \mathbf{1}\{S_A(c) = 1, S_B(c) = 0\}. \quad (21)$$

This identity is simple but important: a net success gain can hide substantial behavioral churn. Paired repairs and regressions are therefore not an optional diagnostic; they are the decomposition of the aggregate number.

The design implication is that acceleration is a search over constrained implementations:

$$\max_{\mathcal{A}} \text{success}(\pi_{\mathcal{A}}) - \lambda \text{latency}(\pi_{\mathcal{A}}) - \gamma \text{risk}(\pi_{\mathcal{A}}, \pi_{\theta}), \quad (22)$$

subject to available quantization, compilation, precision, and kernel choices. The role of the theory is not to solve this optimization directly, but to identify useful coordinates for search: sensitive action channels, sensitive rollout windows, sensitive modules, and low-margin task slices.

4 Experimental Setup

Model and task. We use the GR00T N1.5 LIBERO long-horizon checkpoint and evaluate on LIBERO-10 [9, 10]. The accepted FP16 baseline matches the official reference result of 38/50 on the standard 5-trial LIBERO-10 run. The evaluated policy predicts action chunks with seven action components: x , y , z , roll, pitch, yaw, and gripper. Unless noted otherwise, experiments use 8 denoising steps. For the final tactic-transfer check, we also evaluate a GR00T N1.7 LIBERO-10 checkpoint on a small held-out set; this is used to test whether the tactic-search workflow, not a specific fixed tactic, carries to a newer checkpoint.

Three evidence tracks. The experiments follow the paper’s argument rather than a single backend implementation. Track A is the sensitivity-identification track. It uses controlled action perturbations, first-divergence traces, and `torch.compile` action-head boundaries to locate which action channels, rollout phases, and model boundaries are closed-loop sensitive. Its purpose is to turn the theory into an actionable sensitivity map. Track B is the tactic-search track. It keeps weights in FP16, compiles the action head for speed, and inserts small eager islands or duration windows as protected regions. Its purpose is to test whether the sensitivity map can guide faster implementations with lower paired regression risk. Track C is the held-out selection track. It evaluates the discovered tactic frontier on additional N1.5 folds and on a GR00T N1.7 active-search sequence: warm-start duration-window probes, one CLSG-BO proposal batch, and a new-initialization held-out validation. Its purpose is to test the reusable CLSG-TS workflow rather than one fixed fallback window.

Evaluation platform and timing. All closed-loop rollouts and serving measurements run on a remote single-GPU NVIDIA RTX 5090 machine with LIBERO in headless EGL mode. The policy runs as an inference server and the LIBERO evaluator as a client process on the same machine. Reported latency is server-side warm `get_action` timing; model loading, prewarm, cold `torch.compile`/Inductor compilation, simulator stepping, client/RPC overhead, and video I/O are excluded from the p50 values. A representative FP16 hygiene run used PyTorch 2.8.0+cu128 with CUDA 12.8; over 10,000 server requests on a 15-case mixed workload, FP16 `get_action` latency was 154.82/162.22/182.60 ms at p50/p90/p99, with about 5.48 GB allocated and 5.78 GB reserved CUDA memory after prewarm.

Closed-loop protocol. The sensitivity-guided acceleration search starts from a 33-case matched probe spanning tasks 4, 6, and 8 with selected initial states that include easy successes, speed-only regressions, and beneficial speed-only branches. We then validate tactic selection on two additional N1.5 30-case folds over tasks 0–9, using held-out initial states 15–17 and 18–20. For N1.7, we use LIBERO-10 tasks 0, 1, 4, 6, and 8. A 15-case warm-start slice uses initial states 27–29, a BO proposal batch reuses that slice to evaluate additional duration-window tactics, and a 15-case held-out validation uses new initial states 30–32. For paired comparisons, every mode is evaluated on the same task-initialization pairs with deterministic policy seeds. We use paired repair/regression counts, first-divergence traces, EEF drift, server p50 latency, eager-request fraction, worst-fold success, and worst-fold speedup as the primary diagnostics. We use p50 serving latency as the primary speed metric because the prototype compile experiments launch isolated server processes; mean and maximum latency include cold `torch.compile`/Inductor compilation spikes and are not treated as steady-state deployment metrics.

5 Results

The results are organized around CLSG-TS. Section 5.1 estimates where the closed-loop system is sensitive, using action-channel, rollout-duration, real backend-drift replay, layer-boundary, and first-divergence probes. Section 5.2 uses those probes to search acceleration tactics, instantiate a small CLSG-BO search over duration windows, and evaluate the resulting speed-risk frontier. Section 5.3 maps the evidence back to the five theoretical claims.

5.1 Sensitivity Maps: Not All Perturbations Are Equal

CLSG-TS begins by asking which perturbations are closed-loop sensitive. To isolate sensitivity from the implementation details of a particular backend, we ran small controlled perturbation probes on two FP16-success cases: task 4 init 9 and task 6 init 8. These probes should not be read as an acceleration method. They are local interventions that reveal which directions, rollout windows, and module boundaries are fragile enough that a future compilation, fallback, kernel, or precision tactic could matter.

Table 1: Controlled action perturbations on two FP16-success cases. All full-horizon single-channel probes use the same continuous-action perturbation norm, $\|\delta a\|_2 = 0.03$. S/F denotes success/failure and the number of policy requests; horizon failures usually reach 991 requests.

Probe	Window	Successes	Per-case outcome
No perturbation	Full	2/2	Task 4 init 9: S224; task 6 init 8: S649
6D continuous	Full	0/2	F991; F991
x only	Full	1/2	S349; F991
y only	Full	0/2	F991; F991
z only	Full	2/2	S239; S666
Roll only	Full	0/2	F991; F991
Pitch only	Full	0/2	F991; F991
Yaw only	Full	0/2	F991; F991
y only, task 4	Early / middle / late	1/3	0:75 F991; 75:150 F991; 150:225 S219
y only, task 6	Early / middle / late	2/3	0:200 F991; 200:450 S697; 450:700 S752

Table 1 supports two practical claims. First, action dimensions are not equally sensitive. A z perturbation of the same norm is absorbed in both cases, while y and rotation perturbations cause failures in this small probe. Second, rollout time is not equally sensitive. The same y perturbation is damaging early, but late-window perturbations can be absorbed after the task has entered a more stable phase. This maps the first two axes of the sensitivity space: channel and time. It also matches the linearized view in Section 3: the relevant quantity is not only $\|\eta_t\|$, but also the direction of η_t and the future closed-loop gain multiplying it.

We then zoomed in on the fragile task 4 init 9 case to ask how small a perturbation can still matter. Figure 2 sweeps the y -action perturbation magnitude from 10^{-6} to 5×10^{-4} with deterministic policy seeds. The FP16 baseline succeeds in 224 policy requests. Both full-horizon and early-window perturbations remain successful through 10^{-5} , but fail at 3×10^{-5} and above, reaching the LIBERO horizon. Thus the observed first-flip interval is $(10^{-5}, 3 \times 10^{-5}]$ for both windows. This should not be read as a universal numerical tolerance; it is a local basin-boundary measurement for one fragile rollout. Its deployment implication is nevertheless important: an implementation error that looks far below common local operator tolerances can be behaviorally unsafe if it is aligned with a sensitive action channel and injected before a contact-critical phase.

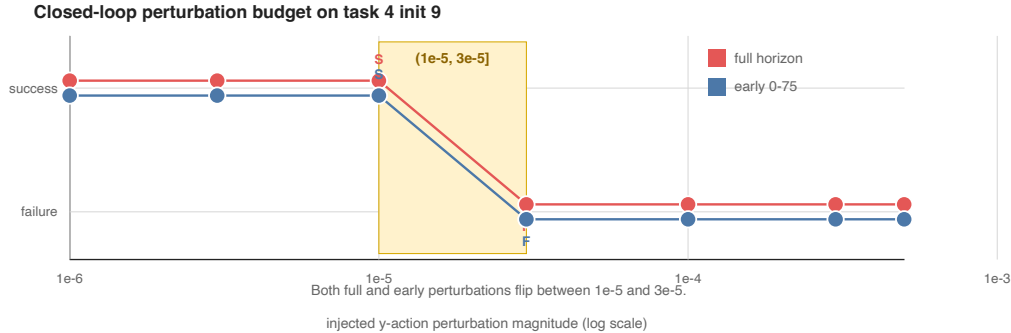


Figure 2: Ultra-low controlled perturbation budget on a fragile FP16-success rollout. The same task 4 init 9 trajectory is perturbed in the y action channel over the full horizon or only during policy steps 0–75. Both curves flip from success to horizon failure between 10^{-5} and 3×10^{-5} action magnitude.

Controlled probes are useful only if they connect to real implementation residuals. We therefore ran a same-observation diagnostic on the two FP16-success cases. Along the nominal FP16 rollout, we queried both the reference policy and an accelerated tactic on the same observation, recorded the per-step backend residual $\eta_t = \pi_\varphi(s_t) - \pi_\theta(s_t)$, and then replayed scaled residual sequences in closed loop:

$$a_t = a_t^{\text{fp16}} + \lambda \eta_t. \quad (23)$$

This is not a production deployment path; it is a directional vulnerability probe that asks whether the actual residuals produced by a tactic are aligned with closed-loop-sensitive directions.

Table 2: Closed-loop replay of real backend drift sequences. Residuals are measured by querying FP16 and the tactic on the same FP16-nominal observations, then replayed as $a_t = a_t^{\text{fp16}} + \lambda \eta_t$. S/F denotes success/failure and policy requests.

Residual source	Case	Window	$\lambda = 0.25$	$\lambda = 0.5$	$\lambda = 1.0$
Speed-only compile	4:9	Full	F991	S221	F991
Speed-only compile	4:9	Early	F991	S224	F991
Speed-only compile	6:8	Full	S496	S833	F991
Speed-only compile	6:8	Early	F991	F991	F991
Blocks 0-3 + 0-120	4:9	Full	S222	S223	S224
Blocks 0-3 + 0-120	4:9	Early	S224	S224	S224
Blocks 0-3 + 0-120	6:8	Full	S229	S632	S464
Blocks 0-3 + 0-120	6:8	Early	S229	S840	S627

Table 2 connects the controlled perturbation budget to actual backend residuals. Speed-only residual sequences can flip FP16-success rollouts, including at $0.25\times$ the measured residual sequence in several windows. The flips are not monotonic in λ ; task 4 init 9 shows failure, success, then failure as the residual scale increases. This is consistent with a high-dimensional, curved trajectory-basin boundary rather than a simple one-sided tolerance. In contrast, the sensitivity-guided protected tactic remains successful for all tested windows and scales up to the unscaled residual sequence. It does not reproduce the FP16 trajectory exactly—task 6 still changes request count substantially—but it lowers outcome-level regression risk. This is the central object that CLSG-TS tries to optimize: not zero residual, but residuals whose direction, timing, and module origin are less dangerous under closed-loop sensitivity.

We also tested whether acceleration-oriented graph boundaries are behaviorally transparent. In this probe, we compile the action-head model while leaving either no block, block 0, or block 1 as an eager island. The variants have almost identical median server latency, but their closed-loop behavior differs sharply.

Table 3: Layer/boundary sensitivity under acceleration-oriented action-head execution. The same two task-initialization pairs are used as in Table 1.

Execution mode	Successes	Task 4 init 9	Task 6 init 8	p50 latency	Speedup
FP16 baseline	2/2	S224	S649	160.53 ms	1.00×
Compile <code>action_head_model</code>	2/2	S222	S916	72.80 ms	2.21×
Compile + block 0 eager	1/2	F991	S404	72.62 ms	2.21×
Compile + block 1 eager	0/2	F991	F991	73.10 ms	2.20×

The full compiled action head is fast and succeeds on both cases, but it is not behaviorally identical to FP16: task 6 takes 916 requests instead of 649. More importantly, making a small eager island does not monotonically improve safety. Block 0 eager fails task 4 but succeeds task 6 faster than the full compiled variant, while block 1 eager fails both. Because all three accelerated variants have nearly the same p50 latency, the result cannot be explained by speed alone. It is a layer/boundary sensitivity effect: changing where numerical or graph-boundary differences enter the policy can move a rollout into a different basin.

Together, the channel, time, and layer-boundary probes form the sensitivity map used by CLSG-TS. These two FP16-success cases do not define a global ranking. Rather, they show that closed-loop risk is structured enough to guide what should be compiled, protected, routed, or left in a more conservative execution path.

Finally, we compared closed-loop traces to locate the first divergence. Table 4 summarizes direct comparisons from the accelerated runs. The main pattern is temporal separation: action differences appear first, millimeter-scale state differences appear later, and outcome flips occur much later through accumulated contact and progress changes.

Table 4: First-divergence diagnostics for accelerated variants. Position thresholds are measured between matched simulator states; action thresholds use continuous-action coordinates.

Case	Pair	Outcome	First action > 0.01	First state > 1 mm	Gripper mismatch
Task 4 init 9	Full → blk0-eager	S222 → F990	Step 21	Step 59, 1.186 mm, z	Step 99
Task 4 init 9	Full → blk1-eager	S222 → F990	Step 8	Step 59, 1.012 mm, x	Step 97
Task 6 init 8	Full → blk0-eager	S916 → S404	Step 8	Step 58, 1.041 mm, x	Step 327
Task 6 init 8	Full → blk1-eager	S916 → F990	Step 53	Step 58, 1.133 mm, y	Step 261

These first-divergence traces explain why latency-equivalent implementations can have different closed-loop outcomes. The decisive failure is rarely a single large action jump. Instead, small early action differences create a delayed state separation; depending on the task margin and contact phase, that separation is either absorbed, repaired into a shorter trajectory, or amplified into horizon failure. This supports a stronger diagnostic principle: for VLA acceleration, one should map sensitivity across action channels, rollout phases, and module boundaries before choosing what to compile, protect, route, or replace.

5.2 Sensitivity-Guided Acceleration Search

The previous probes identify sensitive coordinates; the next question is whether they can guide an engineering choice. We therefore expanded the proxy-guided acceleration search to a 33-case matched set. This set reuses the same task-initialization pairs for every candidate and includes speed-only regressions, baseline successes, and beneficial speed-only branches. The candidates are deliberately simple: compile the action head for speed, then protect selected layer blocks or rollout durations by falling back to eager execution.

Table 5: Layer and duration proxy search on the 33-case matched set. Server p50 is warm serving latency. Eager fraction is the fraction of server requests routed through the protected eager fallback when applicable.

Mode	Protected region	Success	Server p50	Speedup	Eager frac.
FP16 baseline	none	19/33	156.22 ms	1.00×	–
Speed-only compile	none	16/33	70.20 ms	2.23×	–
Blocks 0–3 eager	early action-head blocks	18/33	67.58 ms	2.31×	–
Duration 0–250	broad early rollout window	18/33	88.75 ms	1.76×	0.376
Duration 0–120	narrow early rollout window	19/33	69.66 ms	2.24×	0.190
Duration 0–180	early prefix	18/33	77.55 ms	2.01×	0.281
Duration 0–220	longer early prefix	16/33	90.49 ms	1.73×	0.328
Duration 80–240	contact-centered window	14/33	75.98 ms	2.06×	0.214
Duration 120–280	grasp/lift-centered window	16/33	73.48 ms	2.13×	0.216
Duration 160–240	narrow contact window	15/33	69.29 ms	2.25×	0.111
Duration 240–320	post-grasp window	17/33	66.71 ms	2.34×	0.084

Table 5 is the main discovery probe. Speed-only compilation gives the expected latency gain but loses three successes relative to FP16. A layer-based proxy, blocks 0–3 eager, recovers two of those successes while preserving speed. A coarse duration proxy, steps 0–250, also reaches 18/33 but costs substantially more latency. The finer duration sweep reveals the useful structure: protecting only steps 0–120 restores FP16-level success, 19/33, while preserving speed-only latency. Longer eager windows are not safer. This rules out the simple explanation that more high-precision execution is always better. The sensitive window is narrow and early, but this table should be read as tactic discovery rather than as a final universal selection result.

Table 6: Paired repair/regression decomposition versus speed-only compilation on the 33-case set.

Mode	Repair	Regress	Net	Repaired cases	Regressed cases
Duration 0–250	4	2	+2	4:6, 6:0, 6:6, 8:7	6:7, 6:9
Duration 0–120	7	4	+3	4:6, 6:0, 6:3, 6:6, 8:4, 8:7, 8:10	6:7, 6:9, 6:10, 8:9
Duration 0–180	5	3	+2	4:6, 6:0, 6:6, 8:3, 8:6	6:7, 6:8, 6:9
Duration 0–220	4	4	0	4:6, 6:0, 6:6, 8:10	6:7, 6:8, 6:9, 8:9
Duration 80–240	3	5	-2	6:0, 6:6, 8:3	4:10, 6:7, 6:9, 6:10, 8:8
Duration 240–320	2	1	+1	8:3, 8:10	8:9

Table 6 explains why aggregate success alone is insufficient. The 0–120 window has the best net repair relative to speed-only, but it is not behaviorally identical to FP16. It repairs seven speed-only failures, including cases that both FP16 and speed-only fail, but it also loses four speed-only successes. Thus the accelerated implementation is not simply restored to the FP16 point; it explores a nearby closed-loop solution space.

Table 7: Selected trace diagnostics for speed-only versus the 0–120 duration proxy. The action threshold is $\ell_2 > 0.05$ in action coordinates; EEf thresholds are Euclidean position differences.

Case	Speed	0–120	First action	First EEf > 1 cm	First EEf > 5 cm	Interpretation
4:6	F990	S241	35	72	191	Returns to a short baseline-like success basin.
6:0	F990	S205	92	121	192	Returns to a baseline-like success basin.
6:6	F990	S289	86	99	136	Repairs via a longer successful branch.
8:10	F990	S765	145	154	348	Repairs a speed-only failure through a long branch.
6:3	F990	S376	54	101	299	Discovers a new success branch not found by FP16.
8:4	F990	S982	155	622	628	Near-horizon new success; visible EEf split appears late.
6:10	S225	F990	47	64	212	Main regression; early branch choice breaks a fast success.
8:9	S624	F990	52	106	170	Loses a beneficial speed-only branch.

The trace diagnostics show two mechanisms. For task 4 init 6 and task 6 init 0, the 0–120 policy stays close to a baseline-like basin; the EEf separation from baseline remains small until late in the successful rollout. For task 6 init 3 and task 8 init 4, the same early eager window produces new successful branches that neither FP16 nor speed-only found. The regressions are the complementary effect: beneficial speed-only branches are also basin choices, and a protection policy can erase them. This is why the guide must optimize paired repair/regression and latency together, not blindly minimize distance to FP16.

We then asked whether the local winner on the 33-case probe transfers to held-out initializations. Table 8 summarizes two 30-case validation folds over all LIBERO-10 tasks. Fold A uses initial states 15–17 and Fold B uses initial states 18–20. The combined tactic protects both early action-head blocks 0–3 and policy steps 0–120. Regressions are counted against the FP16 outcome on the same task-initialization pair.

Table 8: Multi-fold tactic selection on two held-out 30-case folds. A single local winner does not dominate all folds, so the selection objective must include worst-fold behavior and paired regressions in addition to speed.

Tactic	Fold A	Fold B	Pooled	Worst success	Mean speedup	Worst speedup	Regressions
Speed-only compile	25/30	20/30	45/60	0.667	2.01×	1.75×	7
Duration 0–120	22/30	25/30	47/60	0.733	1.84×	1.71×	3
Blocks 0–3 + duration 0–120	22/30	25/30	47/60	0.733	1.41×	1.07×	1

The held-out folds change the interpretation. On Fold A, speed-only compilation has the highest aggregate success, showing that an implementation perturbation can also discover beneficial branches. On Fold B, the same speed-only tactic drops to 20/30 and creates five regressions relative to FP16, while the 0–120 and combined tactics match the FP16 aggregate. The combined tactic is the most behavior-conservative candidate, with only one total paired regression across 60 cases, but it gives up speed. The 0–120 tactic is therefore preferable under a speed-constrained objective, whereas the combined tactic is preferable under a behavior-first objective. This is the practical meaning of sensitivity-guided acceleration search: the output is a Pareto choice under a validation protocol, not a proof that one protected region is universally optimal.

We repeated the final selection logic on a GR00T N1.7 checkpoint using a small active-search sequence over duration-window tactics. This is a restricted proof of concept for CLSG-BO, not a full backend optimizer. Phase40 uses a 15-case warm-start slice over tasks 0, 1, 4, 6, and 8 with initial states 27–29. It compares FP16, speed-only compilation, and hand-chosen request windows. Phase41 fits a lightweight RBF surrogate over duration-window parameters and evaluates one BO-selected batch on the same slice. Phase42 then validates the most important candidates on new initial states 30–32. The search space is deliberately narrow: a tactic `window_s.e` runs policy requests $[s, e]$ through conservative eager fallback and the rest through the compiled action head.

Table 9: GR00T N1.7 duration-window CLSG-BO and held-out validation. Phase40 is a warm-start slice using initial states 27–29. Phase42 is a new-initialization held-out slice using initial states 30–32. Server p50 latency is the primary speed metric; repairs and regressions are paired against FP16 on the same cases.

Deployment point	Success	p50 latency	Speedup	Repairs	Regressions
Phase40 FP16 baseline	13/15	85.97 ms	1.00×	—	—
Phase40 speed-only	12/15	61.99 ms	1.39×	1	2
Phase40 window 0–20	14/15	76.30 ms	1.13×	1	0
Phase41 window 2–12	14/15	67.50 ms	1.27×	2	1
Phase42 FP16 baseline	13/15	85.42 ms	1.00×	—	—
Phase42 speed-only	13/15	64.25 ms	1.33×	1	1
Phase42 window 0–20	15/15	80.99 ms	1.05×	2	0
Phase42 window 2–12	14/15	65.76 ms	1.30×	1	0
Phase42 window 4–9	12/15	62.36 ms	1.37×	1	2
Phase42 window 6–11	13/15	63.26 ms	1.35×	2	2

This active-search check reinforces the same lesson and makes the search protocol more explicit. In Phase40, the conservative 0–20 window is the behavior-first point: it improves success to 14/15 with no paired regressions. Phase41 then uses the warm-start observations to select additional duration windows; among them, 2–12 gives a faster speed–risk trade-off. On the new Phase42 held-out slice, 0–20 reaches 15/15 with 2 repairs and 0 regressions, while 2–12 reaches 14/15 with 1 repair, 0 regressions, and a $1.30\times$ p50 speedup. Narrower BO proposals such as 4–9 and 6–11 are faster but reintroduce regressions. Thus BO is useful as a candidate generator, but it does not replace held-out paired validation. The correct output is a frontier: choose 0–20 when regressions are unacceptable, and choose 2–12 when a speed constraint is active.

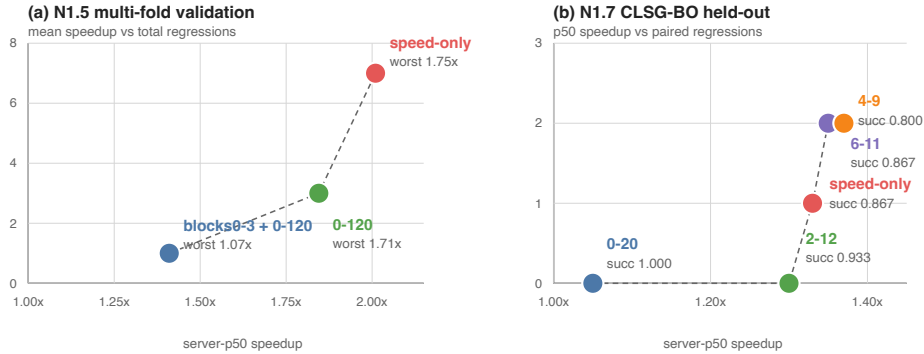


Figure 3: Speed–risk Pareto view of tactic selection. Left: N1.5 multi-fold validation shows that speed-only execution is fastest but has the largest paired regression count, while protected tactics trade speed for behavior. Right: N1.7 CLSG-BO held-out validation shows a behavior-first 0–20 tactic and a faster speed-constrained 2–12 tactic, while narrower BO proposals reintroduce regressions.

5.3 Empirical Support for the Theoretical Claims

The theoretical claims in Section 3 are not used as proof that one accelerated implementation is superior. They organize what must be measured. The experiments support the claims in the following sense.

Table 10: How the empirical results instantiate the perturbation claims.

Claim	Experimental evidence	Interpretation
Claim 1: sensitivity propagation	First-divergence traces show small action differences appearing before millimeter-scale state separation; outcome changes occur much later through accumulated contact and progress changes.	The effect depends on where the perturbation enters the trajectory, not only on its norm.
Claim 2: margin crossing	Duration 0–120 repairs 7 speed-only failures but also regresses 4 speed-only successes on the same 33-case set. Trace diagnostics show both short-basin repairs and lost beneficial branches.	The same tactic can push low-margin failures into success basins while pushing other trajectories across success boundaries.
Claim 3: open-loop insufficiency	Latency-equivalent eager-island variants differ from 2/2 success to 0/2 on the same matched cases, and speed-only compilation is best on Fold A but unsafe on Fold B.	Local numerical or speed checks do not determine closed-loop robustness because the implemented policy induces its own state distribution.
Claim 4: anisotropic sensitivity	Equal-norm action probes differ by channel and phase; an ultra-low y -perturbation sweep flips task 4 init 9 between 10^{-5} and 3×10^{-5} action magnitude in the full and early windows. Real speed-only backend residual replay causes multiple S→F flips, while the protected residual sequence remains successful up to $1.0\times$ on the same cases. Acceleration variants with similar p50 latency differ from 2/2 success to 0/2 depending on layer boundary. In the 33-case search, duration 0–120 reaches 19/33 at 69.66 ms, while 0–220 drops to 16/33 at 90.49 ms. Across held-out folds and the N1.7 CLSG-BO check, speed-only, duration windows, combined protection, and BO proposals have different repair/regression profiles.	Perturbation risk is structured by action direction, future closed-loop gain, task margin, module location, and roll-out phase. More protection is not necessarily safer.
Claim 5: solution-space exploration	Duration 0–120 restores FP16-level aggregate success on the probe, but does so by repairing 7 speed-only failures and regressing 4 speed-only successes. On held-out folds, speed-only is best on one slice and unsafe on another; on N1.7, <code>window_0_20</code> is behavior-first while CLSG-BO finds <code>window_2_12</code> as a faster speed-constrained point.	The goal is not to recover the FP16 point exactly or to trust one local winner. Acceleration searches a nearby implementation space where some perturbations discover better basins and others destroy beneficial branches, so tactics and active-search proposals must be validated across slices.

6 Analysis

The experiments test one closed-loop object: a post-training implementation map that perturbs the action function. Compiled action-head execution, eager islands, duration fallbacks, active-search proposals, and potential tactic routers all change where and when implementation differences enter the controller. These differences can be small under local numerical or latency checks, yet still create delayed trajectory branching

under closed-loop execution. The backend changes, but the evaluation target remains the same: which perturbations are absorbed, which cross task margins, and which tactics give acceptable speed–risk trade-offs.

Why can acceleration improve a rollout? An accelerated policy is not guaranteed to be worse than FP16 in a finite simulator benchmark. FP16 is not an oracle; it is a learned policy with its own fragile regions. A small implementation perturbation can move a trajectory away from a failure mode, especially when the FP16 policy is already weak on a task slice such as LIBERO task 8 in our baseline. Therefore, an aggregate improvement should not be read as “the accelerated implementation is inherently better.” It means the perturbed implementation enters different trajectory basins, some of which are more favorable under the benchmark’s finite initial states and success predicates.

Why can local validation fail to predict closed-loop behavior? Local validation is measured on fixed observations, fixed shapes, or fixed operator boundaries. Closed-loop rollouts produce observations adaptively. Once an accelerated action changes the environment state, later observations may differ from the reference distribution. This creates two mismatches: the local action error estimate may not apply to the new states, and the success function may be effectively discontinuous around contact and placement boundaries. This is why latency-equivalent or locally similar implementations can have different rollout outcomes.

A control-theoretic interpretation. From a feedback-control perspective, post-training acceleration acts like a structured input disturbance injected into the learned controller. We can write an accelerated action as

$$a_t^\varphi = \pi_\theta(s_t^\varphi, l) + \eta(s_t^\varphi, l), \quad (24)$$

where η denotes the action difference induced by the accelerated execution path. Linearizing around an FP16 trajectory gives

$$\delta s_{t+1} \approx (A_t + B_t K_t + B_t \Delta K_t) \delta s_t + B_t \eta_t, \quad (25)$$

where $A_t = \partial f / \partial s$, $B_t = \partial f / \partial a$, $K_t = \partial \pi_\theta / \partial s$, and $\Delta K_t = \partial \pi_\varphi / \partial s - \partial \pi_\theta / \partial s$ along the trajectory. The closed-loop sensitivity is therefore governed not only by the magnitude of η_t , but also by the local closed-loop gain induced by the environment dynamics and the policy feedback. A small action perturbation can be attenuated in low-gain regions and amplified near contact, grasping, or placement transitions.

This also makes success a margin problem. Let $h(s_T)$ denote an implicit terminal success margin, with success when $h(s_T) > 0$. A rollout with large positive margin can tolerate sizeable action perturbations, while a low-margin rollout can flip if acceleration moves $h(s_T)$ across zero. This explains why paired outcomes contain both repairs and regressions: an accelerated implementation can move one trajectory away from a failure basin while pushing another trajectory across a success boundary. The useful diagnostic is therefore not just average action error, but the alignment between action perturbations, closed-loop sensitivity, and task success margins.

What should be reported for VLA acceleration? The minimum evidence package should include local action drift or numerical agreement, closed-loop success on matched task-initialization pairs, per-task and per-init success rates, paired repair/regression counts, latency and memory under the same protocol, first-divergence or EEF drift diagnostics for representative flips, and an explicit distinction between prototype tactic search and final production deployment. Without these pieces, a result can look like a speedup or a success-rate gain while hiding substantial closed-loop redistribution.

7 Implications

Evaluation. VLA inference acceleration should be evaluated as a closed-loop policy perturbation. Reporting only MSE, aggregate success, or latency can be misleading because each metric can hide a different failure mode. Paired rollout flips are a compact way to expose whether a faster implementation is a monotonic improvement or a redistribution of successes.

A practical guide. The experiments suggest a simple workflow for future VLA acceleration studies:

1. Build a speed-oriented candidate, such as low-precision execution, a compiled graph, or a fused kernel path, and measure both latency and matched closed-loop success.
2. Decompose the result into paired repairs and regressions instead of relying on aggregate success alone.
3. For the fragile cases, run first-divergence traces and controlled probes to identify sensitive action channels, rollout windows, and model or graph boundaries.
4. Protect the smallest candidate region—a layer island, duration window, action correction, or high-precision fallback—that improves the repair/regression balance on the probe set.
5. Re-evaluate candidate tactics on multiple held-out matched folds, because a proxy that works on one slice can overfit and destroy beneficial branches elsewhere.
6. Select a deployment tactic from the Pareto frontier using explicit constraints such as worst-fold success, maximum paired regressions, and minimum speedup.

This converts acceleration from a search for a single numerically closest implementation into an iterative exploration of the closed-loop solution space. We call the resulting workflow Closed-Loop Sensitivity-Guided Tactic Search (CLSG-TS), summarized in Algorithm 1.

Algorithm 1: Closed-Loop Sensitivity-Guided Tactic Search (CLSG-TS)

1. **Input:** candidate tactics $\mathcal{T}$, matched probe rollouts $\mathcal{P}$, measured latency $L(T_i)$, and paired repairs/regressions $R^+(T_i), R^-(T_i)$ on each validation fold.
2. Run cheap filters on each $T_i \in \mathcal{T}$: operator coverage, expected speedup, static shape compatibility, open-loop drift, layerwise drift, action-channel drift, and whether the tactic touches known contact- or basin-sensitive regions.
3. Evaluate the surviving tactics on a small matched probe set containing fragile successes, fragile failures, and speed-only branch changes. Record success, latency, first divergence, EEF drift, repairs, and regressions.
4. Estimate closed-loop risk using paired regressions, worst-case fragile failures, and sensitivity diagnostics; keep a small candidate frontier rather than a single probe winner.
5. Optionally fit a lightweight surrogate over structured tactic parameters, such as fallback-window start/end, and use an acquisition rule to propose the next small batch of candidates.
6. Re-score the frontier on held-out matched folds. Select a behavior-first tactic by maximizing worst-fold success and minimizing paired regressions, or select a speed-constrained tactic by maximizing speedup subject to explicit behavior constraints. If no single tactic dominates, evaluate conditional tactic selectors over task or trajectory regimes.
7. **Output:** a behavior-first tactic, a speed-constrained Pareto tactic, or a validated tactic selector, with its matched success, regression, and latency report.

Figure 4: Closed-Loop Sensitivity-Guided Tactic Search. The procedure is not an optimal-control solver; it is a practical way to spend limited rollout budget on tactics most likely to matter under closed-loop sensitivity.

Sensitivity-guided implementation search. This view is analogous to compiler tactic search, but with a different validation cost. A system such as TensorRT can enumerate many operator implementations for a fixed shape, benchmark their latency, check local numerical agreement, and choose the fastest valid tactic. For VLA acceleration, the candidate space is similar in spirit: quantization scopes, graph boundaries, eager islands, fallback windows, kernel replacements, calibration choices, and tactic routers are all implementation tactics. The difference is that the validity test is not only local numerical agreement. It is paired closed-loop

behavior under a rollout budget. Exhaustively evaluating every candidate with robot rollouts is therefore infeasible.

The practical replacement is sensitivity-guided search rather than brute-force search. Cheap static and open-loop filters should first prune candidates by latency benefit, operator coverage, layerwise drift, action-channel drift, attention-routing sensitivity, and whether the candidate touches contact-critical or basin-critical regions. A smaller set can then be evaluated on proxy rollouts containing fragile matched cases, using first-divergence, EEf drift, and paired repair/regression. Only the most promising candidates should receive larger held-out closed-loop validation. In this sense, the 0–120 duration proxy is not a universal tactic; it is a candidate discovered by sensitivity-pruned search. Table 8 shows the next step: after discovery, the tactic must be re-scored across validation folds, where behavior-first and speed-constrained objectives may choose different points on the Pareto frontier. Table 9 further shows that a lightweight BO loop can propose new duration-window candidates on a newer checkpoint, but that these proposals still require held-out paired validation. The reusable object is therefore the search protocol, not the literal 0–120 or 0–20 window.

Active tactic search and routing. The N1.7 experiment suggests a design implication: a deployment need not rely on one manually chosen acceleration tactic. In our restricted pilot, BO searches only over request-window start and end. The same protocol can in principle search layer islands, precision profiles, compile targets, or conditional tactic routers. This is reminiscent of mixture-of-experts routing, but the routed objects are not model experts or learned parameters. The selector chooses inference tactics such as eager execution, speed-only compiled execution, or sensitivity-preserving fallback windows. In this view, acceleration becomes closed-loop-aware inference scheduling. The held-out result also shows the risk: a surrogate-selected or probe-selected tactic can overfit to initial states, so any search proposal or router must be evaluated by paired held-out rollouts rather than assumed safe.

This selection should be coarse-grained in a production runtime. Task-level tactic choice, episode-level profiles, or a small number of fixed phase windows can be implemented by pre-building and pre-warming a few execution profiles. In contrast, per-step, per-layer, or per-operator tactic switching can destroy the assumptions that make TensorRT engines, CUDA Graph replay, and fused kernels fast: stable shapes, stable graph structure, persistent execution contexts, and predictable memory allocation. Thus a sensitivity map should usually decide which profile to use or which broad phase to protect, not trigger arbitrary fine-grained graph changes at runtime.

Calibration and protection. Calibration or acceleration planning should not only minimize average layerwise error or maximize local speed. It should consider which errors matter for downstream control. A useful calibration or profiling set should cover fragile contact states, low-margin success boundaries, and task slices where the FP16 policy is already weak. A control-aware objective would approximate Equation 13: weight errors by estimated closed-loop sensitivity, not only by their open-loop magnitude. This points toward rollout-informed calibration sets, residual action correction, sensitivity-guided compile boundaries, or risk-gated precision fallback as natural next steps.

Closed-loop credit assignment. This perspective borrows an idea from reinforcement learning without requiring full policy optimization: local action errors should be assigned credit according to their downstream effect on future rollout outcomes. In standard PTQ or speed-only compilation, an error η_t is usually priced by its immediate norm or ignored if latency improves. In the closed-loop view, its risk is closer to $\eta_t^\top G_t \eta_t$, which assigns larger cost to directions that propagate through dynamics and reduce terminal task margin. This suggests a lightweight alternative to direct RL: use paired rollouts, first-divergence states, low-margin states, or repair/regression cases to estimate sensitivity proxies, then guide calibration, mixed precision, residual correction, compile boundaries, kernel replacement, or fallback decisions. The 0–120 duration proxy in Table 5 is an example of a useful proxy discovered by this process; the multi-fold comparison in Table 8 shows why it should still be selected under explicit speed-risk constraints rather than treated as universally optimal.

Sensitivity-guided acceleration. For VLA or world-model deployment, layer selection, graph compilation, and fallback windows should not be based only on parameter count, FLOPs, compiler convenience, or latency.

A useful acceleration candidate should have both high compute benefit and low closed-loop sensitivity. Informally, for a module or protected region i with expected speed benefit g_i and acceleration-induced perturbation η_i , the relevant risk is closer to a sensitivity-weighted quantity,

$$r_i \approx \mathbb{E} [\eta_i^\top G_i \eta_i], \quad (26)$$

where G_i summarizes downstream dynamics, feedback, and task-margin sensitivity. This suggests a mixed-precision and mixed-execution selection rule: prioritize regions with large g_i/r_i , and leave routing-sensitive, contact-critical, basin-critical, or margin-critical regions in higher precision or eager execution unless closed-loop tests show otherwise.

The acceleration search illustrates both sides of this rule. Compiling the entire action head maximizes warm serving speed but increases closed-loop regressions. Protecting a broad early duration, 0–250, improves behavior but gives up too much latency. Protecting only 0–120 finds a better speed-constrained point in the design space, while adding early-block protection further reduces paired regressions at additional latency cost. On N1.7, a 0–20 request window is behavior-first, while the BO-discovered 2–12 request window is a faster speed-constrained point. The deployment lesson is therefore to treat mixed precision and fallback routing as constrained search over speed benefit and closed-loop risk, not as a one-shot numerical equivalence transform.

Fine-tuning. Acceleration-aware fine-tuning is policy adaptation under deployment constraints, including low precision, compiled execution, or custom kernels. If the objective is only behavior cloning on static observations, it may still miss closed-loop state distribution shift. More policy-aware objectives may be needed, including rollout-informed calibration, DAGger-style data aggregation, or task-slice reweighting.

Deployment claims. Our results are behavior-level evidence for prototype action-head compilation, fallback, and routing tactics. They do not establish final production latency, memory, or energy improvements. A complete deployment claim requires production compile/graph backends or custom kernels, plus end-to-end inference profiling under the same closed-loop protocol. High-performance runtimes such as TensorRT, CUDA Graph replay, or custom C++/CUDA operators impose constraints that eager PyTorch does not: graph breaks, dynamic routing, or duration-dependent fallback may add scheduling overhead, memory pressure, graph-cache churn, or lost fusion. A practical implementation should therefore realize CLSG-TS decisions as a small set of stable, pre-warmed deployment profiles, for example a conservative profile for sensitive phases and an aggressive profile for high-margin phases, and measure whether the profile-switching overhead is smaller than the recovered speedup. CLSG-TS is therefore a backend-selection and validation guide; the final backend must still realize the selected tactic with stable graphs and measured end-to-end cost.

8 Limitations

This study has several limitations. The mixed-execution acceleration probes use prototype action-head compilation boundaries and duration fallbacks; they should be read as diagnostic evidence for CLSG-TS rather than a production deployment benchmark. The backend gap is especially important for edge deployment on platforms such as NVIDIA Thor: a tactic that is cheap to express in PyTorch may be expensive to realize in TensorRT, CUDA Graph replay, or a custom operator stack if it requires dynamic routing, frequent graph cuts, or multiple warmed execution paths. We therefore claim that closed-loop sensitivity should decide which regions deserve protection, not that the exact PyTorch tactics are the final deployable form.

The empirical scale is also limited. Results are centered on one GR00T N1.5 checkpoint and LIBERO-10, plus a small N1.7 active-search and held-out transfer check. The N1.5 acceleration track uses a 33-case discovery probe and two 30-case held-out folds. The N1.7 sequence uses 15-case warm-start and BO-proposal slices plus a 15-case held-out validation over tasks 0, 1, 4, 6, and 8. These experiments expose tactic instability and speed–risk structure, but they are not a definitive deployment benchmark or open-world robustness claim. The current BO instantiation searches only duration-window tactics; extending it to layer boundaries, precision profiles, production runtime tactics, online sensitivity-triggered routing, broader model families, real-robot experiments, and more exhaustive geometric diagnostics remains future work.

These limitations do not weaken the central methodological point: local action approximation and closed-loop policy behavior are distinct evaluation targets.

9 Conclusion

Post-training inference acceleration of VLA policies should be viewed as closed-loop policy perturbation. Even small action errors can matter because they enter a feedback loop, alter the induced state distribution, and interact with task-dependent success margins. Prototype action-head compilation gives substantial warm serving speedups, yet speed-only execution introduces closed-loop regressions. CLSG-TS finds useful protected regions, such as the early 0–120 policy-step window, but held-out validation shows that the final choice must be made as a constrained tactic-selection problem. On N1.7, a duration-window CLSG-BO pilot further shows how active search can propose a speed-constrained tactic, 2–12, while held-out validation keeps the behavior-first 0–20 tactic separate from faster but riskier proposals.

The main lesson is practical: VLA acceleration papers should report paired closed-loop rollouts, re-pair/regression decompositions, sensitivity probes, held-out tactic validation, and latency under the same protocol, not only offline drift, aggregate success, or raw serving speed. Small action errors matter when they change the trajectory basin. Useful acceleration therefore comes from finding which perturbations are closed-loop sensitive and selecting the smallest protective tactics that satisfy explicit speed–risk constraints. The same sensitivity maps also suggest a future training route: noise-aware distillation or reinforcement-learning-based robust fine-tuning can focus perturbation training on the states, durations, action channels, and graph boundaries that matter most in closed loop, rather than injecting noise uniformly.

References

- [1] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. URL <https://fbsbook.org>.
- [2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. URL <https://arxiv.org/abs/2212.06817>.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. URL <https://arxiv.org/abs/2307.15818>.
- [4] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL <https://arxiv.org/abs/2303.04137>.
- [5] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022. URL <https://arxiv.org/abs/2208.07339>.
- [6] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. URL <https://arxiv.org/abs/2210.17323>.
- [7] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Pannag Sanketi, Quan Vuong, Thomas Kollar, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. URL <https://arxiv.org/abs/2406.09246>.
- [8] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023. URL <https://arxiv.org/abs/2306.00978>.
- [9] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023. URL <https://arxiv.org/abs/2306.03310>.
- [10] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llonet, Loic Magne, Ajay Mandlekar, Avnish Narayan,

- Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [11] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *arXiv preprint arXiv:1011.0686*, 2010. URL <https://arxiv.org/abs/1011.0686>.
- [12] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022. URL <https://arxiv.org/abs/2211.10438>.
- [13] Jingxuan Zhang, Yunta Hsieh, Zhongwei Wan, Haokun Lin, Xin Wang, Ziqi Wang, Yingtie Lei, and Mi Zhang. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026. doi: 10.48550/arXiv.2602.20309. URL <https://arxiv.org/abs/2602.20309>.